Moving to Linked List

layvathna@gmail.com

Objective

By the end of this lesson, students will be able to:

- Recall the limitations of arrays for dynamic insertions and deletions.
- Explain how pointers can be used to connect data elements dynamically.
- Build a simple chain of elements using pointers.
- Recognize this structure as a Linked List and explain its advantages.
- Compare array vs. linked list in terms of time and space complexity.

Recall

You are managing a student registration list. New students often arrive, some drop out, and sometimes you need to rearrange them.

- With arrays:
  - Insert at front → O(n) (shifting needed)
  - Delete in the middle → O(n) (shifting needed)
  - Resize when full → expensive copy operation

- “Is this efficient if students keep joining and leaving?”

The idea

- Example:
  - [Student1] → [Student2] → [Student3]
- If a new Student4 comes in front of Student2:
  - [Student1] → [Student4] → [Student2] → [Student3]
- If Student2 drops out, instead of shifting Student3 forward:
  - [Student1] → [Student3]

How Arrays Work (Index = Connection)

- In arrays, elements are stored side by side in memory.
- We don’t store arrows; we just rely on indexes:

```
Index:   0    1    2    3
Value:  [A]  [B]  [C]  [D]
```

To go from B to C, we simply do index + 1.

The Problem with Deletion

- If we delete B (index 1):

```
Index:   0    1    2    3
Value:  [A]  [ ]  [C]  [D]
```

- Now index 1 is wasted → it stores nothing.
- If we loop over the array, we'll see:

```
A, (empty), C, D
```

- So, the common solution is to shift elements forward:

```
Index:   0    1    2
Value:  [A]  [C]  [D]
```

But shifting every element takes extra time → O(n).

"What if instead of shifting all the elements, we could just tell A to skip B and go directly to C?"

Tell A to skip B and go directly to C

- A always goes to B before C because of index which is the concept of Array

```
Index:   0    1    2    3
Value:  [A]  [ ]  [C]  [D]
```

So if we don’t use array, how can we connect from 1 element to another element?

Array vs Pointer

```
Index:   0     1     2     3
Value:  [ A ] [ B ] [ C ] [ D ]
Addr:   100   104   108   112   (assume 4 bytes each)
```

- In an array,
  - we store one variable (e.g., arr[ ]).
  - That variable holds the base address of the array in memory.
  - Because elements are side-by-side, we can always find the next one by:
    - Next element = base address + (index * size)

- What can we do for pointer?

```
Value:   [A | → ]   [C | → ]   [D | → ]
Addr:     200         312         156
```

Array vs Pointer

- elements are not side-by-side.
- We can’t use index math to jump around.
- Each element stores a pointer to the next element’s address.
- “If every element knows only who comes next… how do we even start the journey?”

```
head → 200: [A | 312] → 312: [B | 156] → 156: [C | NULL]
```

Pointer (LinkedList)

- Just like arrays have a base address, linked lists have a head pointer.
- The head points to the first node’s address.
- From there, we follow pointers step by step.

```
head → 200: [A | 312] → 312: [B | 156] → 156: [C | NULL]
```

LinkedList

- What do we need?
  - A variable that can hold more than 1 thing → Struct
  - A place to start → Head

LinkedList

```
struct Student {
    int id;
    Student* next;
};
```

The first students, will be head of the linkedList.

When the next point to null, we knows that it is the end of the list

Quesitons

- Insertion at Front
- Deletion in Middle
- Jumping Around
- Sequential Access
- Imagine books on a shelf (array) vs. books tied with a string (linked list).
    - Which is easier if you need to:
    - Add a book in the middle?
    - Pick the 7th book directly?

Summary

- Arrays
  - Elements stored side by side in memory.
  - Use index + base address to find the next element.
  - Problem: Deletion creates gaps → shifting needed (O(n)).
- Pointers as the Solution
  - Instead of shifting, we can re-link elements using arrows (addresses).
  - Each element stores: data + pointer to next element.
- Linked List
  - A chain of elements connected by pointers.
  - We only need to store head pointer (like base address in arrays).
  - Insert/Delete = efficient (just update arrows).
- Trade-off: must follow arrows one by one (O(n) to access).